



POLITECNICO
MILANO 1863

Politecnico di Milano

Corso di Laurea in Ingegneria Informatica

Relazione Progetto di Reti Logiche

Docente: Prof. Gianluca Palermo

Studentesse:

Isabel Reguera Sierva
Alice Piacentini

Matricola: 210936
Matricola: 210253

Anno Accademico 2024/2025

Indice

1	Introduzione	3
1.1	Descrizione	3
1.2	Funzionamento	3
1.3	Entity del componente	4
1.4	Descrizione della memoria	5
1.4.1	Struttura della memoria	5
1.4.2	Accesso alla memoria	5
2	Architettura	6
2.1	Segnali Interni	6
2.2	Descrizione della FSM	7
2.2.1	Idle State	7
2.2.2	S ₁ State	8
2.2.3	S ₂ State	8
2.2.4	S ₃ State	8
2.2.5	S ₄ State	8
2.2.6	S ₅ State	8
2.2.7	S ₆ State	8
2.2.8	S ₇ State	9
2.2.9	S _{7_wait} State	9
2.2.10	S ₈ State	9
2.2.11	S _{9_write} State	9
2.2.12	S ₁₀ State	9
2.2.13	S ₁₁ State	10
2.2.14	S ₁₂ State	10
2.2.15	S _{12_wait} State	10
2.3	Descrizione dei processi	10
2.3.1	Processo 1	10
2.3.2	Processo 2	10
2.3.3	Processo 3	10
2.3.4	Processo 4	10

3	Risultati sperimentali	13
3.1	Test bench	13
3.2	Report di sintesi	15
4	Conclusioni	17

Introduzione

1.1 Descrizione

Lo scopo del progetto è quello di implementare un modulo hardware (HW) in VHDL che si interfacci con una memoria dove sono memorizzati i dati e dove andrà scritto, dopo l'applicazione di un filtro differenziale, il risultato della sequenza di parole elaborata.

Il filtro differenziale è il seguente:

$$f'(i) = \frac{1}{n} \sum_{j=-l}^{+l} C_j \cdot f[i+j] \quad (1.1)$$

dove:

- $l = 2$ per il filtro di ordine 3,
- $l = 3$ per il filtro di ordine 5,
- n è il valore di normalizzazione.

1.2 Funzionamento

All'accensione (comportamento garantito dal test bench) o dopo un RESET, il modulo si trova nello stato di *Idle*, in cui tutti i segnali interni e i registri di memoria sono inizializzati a “0”.

Il modulo rimane in attesa dell'ingresso **START**: quando questo sale a “1”, inizia l'elaborazione della sequenza contenuta in memoria. Esso rimane a “1” durante tutta l'elaborazione. Quando la sequenza da elaborare è terminata il modulo porta a “1” il segnale **DONE**, che rimane alto fino a che **START** non torna nuovamente a “0”.

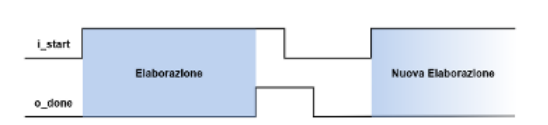


Figura 1.1: Diagramma temporale dei segnali START e DONE

1.3 Entity del componente

Il componente possiede la seguente interfaccia (Figura 1.2).

```
entity project_reti_logiche is
  port (
    i_clk   : in std_logic;
    i_rst   : in std_logic;
    i_start : in std_logic;
    i_add   : in std_logic_vector(15 downto 0);

    o_done  : out std_logic;

    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_data : out std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
  );
end project_reti_logiche;
```

Figura 1.2: Entity del componente progettato

Tutti i segnali sono sincroni e devono essere interpretati sul fronte di salita del clock. Cambiamenti fuori dal rising edge non hanno effetto immediato, eccetto il reset, che è asincrono.

- **i_clk**: segnale di clock in ingresso
- **i_rst**: segnale di reset asincrono
- **i_start**: segnale di avvio generato dal testbench
- **i_add**: indirizzo iniziale della sequenza
- **o_done**: segnale di fine elaborazione
- **o_mem_addr**: indirizzo verso la memoria
- **i_mem_data**: dato letto dalla memoria
- **o_mem_data**: dato da scrivere in memoria
- **o_mem_en**: enable per l'accesso in memoria
- **o_mem_we**: segnale di scrittura (1) o lettura (0)

1.4 Descrizione della memoria

1.4.1 Struttura della memoria

La memoria è organizzata come riportato in Tabella 1.1.

Indirizzo	Contenuto	Dimensione
i_add	K_1	1 Byte
$i_add + 1$	K_2	1 Byte
$i_add + 2$	S	1 Byte
$[i_add + 3 ; i_add + 3 + 13]$	$C_1, \dots, C_{14}$	14 Byte
$[i_add + 17 ; i_add + 17 + k - 1]$	$W_1, \dots, W_k$	k Byte
$[i_add + 17 + k ; i_add + 17 + 2k - 1]$	$R_1, \dots, R_k$	k Byte

Tabella 1.1: Struttura della memoria fornita

1.4.2 Accesso alla memoria

Lettura

Per leggere i dati di interesse dalla memoria si utilizza il segnale **i_mem_data**, che restituisce al componente il dato contenuto in memoria all'indirizzo **o_mem_addr**. Trattandosi di segnali sincroni, per leggere correttamente è necessario impostare:

- **o_mem_en** = 1
- **o_mem_we** = 0
- **o_mem_addr** all'indirizzo di interesse

e attendere il ciclo di clock successivo.

Scrittura

Per scrivere un dato in memoria si utilizza il segnale **o_mem_data**, che indica il valore da scrivere. Trattandosi di segnali sincroni, per scrivere correttamente è necessario impostare:

- **o_mem_en** = 1
- **o_mem_we** = 1
- **o_mem_addr** all'indirizzo di scrittura

e attendere il ciclo di clock successivo.

Architettura

2.1 Segnali Interni

- **k₁**: registro da 8 bit contenente il byte più significativo del valore **k**, ovvero la lunghezza totale della sequenza
- **k₂**: registro da 8 bit contenente il byte meno significativo del valore **k**
- **k**: registro da 16 bit (composto da **k₁** e **k₂**) contenente la lunghezza complessiva della sequenza da elaborare
- **s**: registro da 8 bit contenente un valore di controllo. Il suo LSB (**S(0)**) viene utilizzato per selezionare il filtro da applicare
- **firstAddr**: registro da 16 bit che memorizza l'indirizzo iniziale **i_add**. Sebbene **i_add** sia costante durante l'elaborazione (**START=1**), questo registro è stato usato per aumentare robustezza e leggibilità
- **currAddrW**, **currAddrR**: registri da 16 bit che rappresentano, rispettivamente, l'indirizzo della prima parola da inserire in words e l'indirizzo di scrittura del risultato
- **currState**, **nextState**: segnali che rappresentano lo stato corrente e lo stato prossimo della macchina a stati finiti (FSM)
- **coeff**: array di 7 coefficienti (ognuno rappresentato come valore signed da 8 bit) da utilizzare per l'elaborazione: [**C1**, **C2**, ..., **C7**] se **LSB(s)** = 0, oppure [**C8**, ..., **C14**] se **LSB(s)** = 1
- **words**: array di 7 parole (ognuna rappresentata come signed da 8 bit) lette dalla memoria a partire da **currAddrW**. Non conoscendo a priori il valore di **k**, non era possibile definire un vettore che contenesse tutte le parole della sequenza. Per questo è stato definito come registro interno un vettore di lunghezza fissa pari a 7, in modo da gestire sia il filtro di ordine 3 che quello di ordine 5 in un'unica soluzione. Infatti, nel caso peggiore (filtro di ordine 5), per applicare il filtro a un dato è necessario conoscere i 3 valori precedenti e i 3 successivi ad esso. Analogamente, nel caso di filtro di ordine 3, i valori richiesti (2 precedenti e 2 successivi) sarebbero comunque compresi nei 7.

- **counter_coeff**, **counter_words**: contatori a 3 bit usati per indicizzare gli array **coeff** e **words**
- **i**: registro da 16 bit che rappresenta l'indice corrente della parola in elaborazione ($0 \leq i \leq k - 1$)
- **result**: registro da 8 bit contenente il risultato dell'elaborazione da scrivere in memoria

2.2 Descrizione della FSM

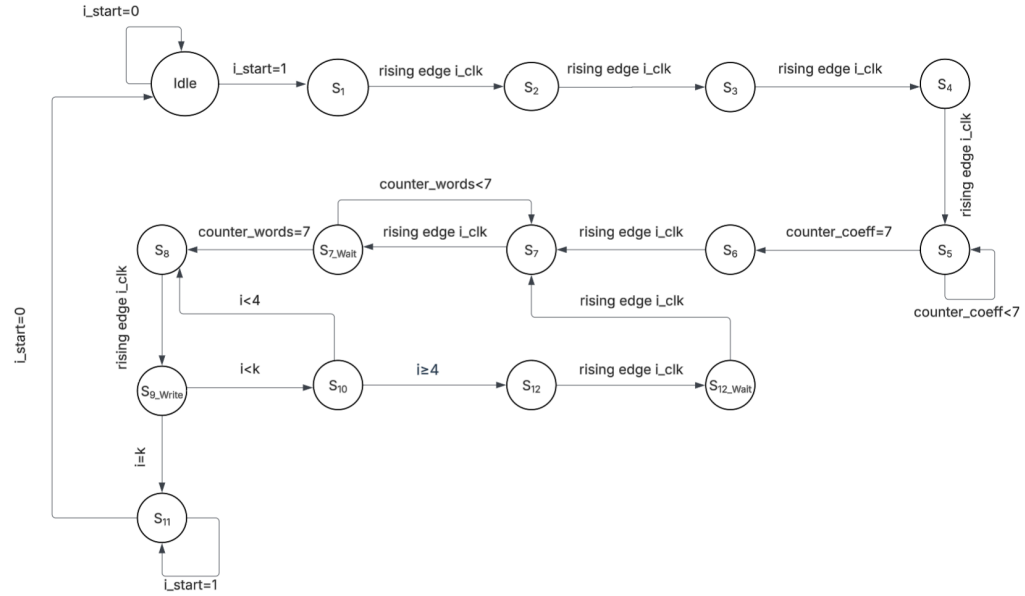


Figura 2.1: macchina a stati

Il modulo è implementato come un automa a stati finiti composto da 15 stati (Idle, S₁, S₂, S₃, S₄, S₅, S₆, S₇, S_{7_Wait}, S₈, S_{9_Write}, S₁₀, S₁₁, S₁₂, S_{12_Wait}) descritti in seguito.

2.2.1 Idle State

È lo stato in cui si trova la FSM al momento dell'accensione e dopo ogni RESET. Qui vengono inizializzati tutti i registri interni al valore 0 di default. Rimane in questo stato finché non riceve uno START.

2.2.2 S₁ State

In questo stato **i_add** viene assegnato al registro **FirstAddr**, che verrà aggiornato al successivo ciclo di clock. Viene preparata la memoria per leggere in **i_mem_data** il valore di **k₁** allo stato successivo: **o_mem_en** = 1 per abilitare l'interazione con la memoria, **o_mem_we** = 0 per leggere e **o_mem_addr** = **i_add**. Queste impostazioni per leggere dalla memoria saranno uguali negli stati di lettura successivi, fatta eccezione per **o_mem_addr** che viene aggiornato.

2.2.3 S₂ State

In questo stato `i_mem_data` viene assegnato al registro `K1`. Si aggiorna `o_mem_addr` a `FirstAddr + 1`.

2.2.4 S₃ State

In questo stato `i_mem_data` viene assegnato al registro `K2`. Si aggiorna `o_mem_addr` a `FirstAddr + 2`.

2.2.5 S₄ State

In questo stato vengono assegnati rispettivamente i valori `i_mem_data` e `FirstAddr + 17` ai registri `S` e `currAddrW`. Si aggiorna `o_mem_addr` a `FirstAddr + 3`.

2.2.6 S₅ State

In questo stato, in base al LSB di **S**, si riempie il vettore **coeff** con i primi/ultimi 7 coefficienti. La FSM rimane in questo stato finché **counter_coeff** < 7. A ogni ciclo in questo stato, **o_mem_addr** viene aggiornato per leggere il nuovo coefficiente al ciclo successivo secondo la formula:

- se $S(0) = 0$:

[illegible]

- se $S(0) = 1$:

```
o_mem_addr ← std_logic_vector(FirstAddr + 4 + 7 +  
                                resize(counter_coeff, FirstAddr'length))
```

2.2.7 S₆ State

In questo stato viene calcolato K e assegnato al corrispettivo registro usando la formula:

$$k \leftarrow (\text{shift_left}(\text{resize}(k1, k'\text{length}), 8)) + \text{resize}(k2, k'\text{length})$$

Viene quindi preparata la memoria per leggere le parole a partire dallo stato successivo, assegnando `currAddrW` a `o_mem_addr`.

2.2.8 S_7 State

In questo stato si riempie il vettore **words** con le 7 parole contenute in memoria a partire dall'indirizzo **currAddrW**. Qui viene incrementato a ogni ciclo il contatore **counter_words**.

2.2.9 S_{7_wait} State

In questo stato viene inviato l'indirizzo del dato successivo alla memoria e si attende il ciclo di clock successivo per avere il nuovo valore disponibile su **i_mem_data**. Se **counter_words** arriva a 7 si prepara semplicemente per S_8 , altrimenti torna in S_7 a salvare la parola.

2.2.10 S_8 State

In questo stato viene aggiornato l'indirizzo in cui scrivere in memoria secondo la formula:

$$\text{currAddrR} \leftarrow \text{FirstAddr} + 17 + k + i$$

Viene inoltre applicato il filtro differenziale all' i -esima parola e il risultato viene assegnato al registro **result**. Si incrementa infine i per passare alla prossima parola da elaborare.

2.2.11 S_{9_write} State

In questo stato viene controllato il valore di i :

- se $i < k$, si passa allo stato S_{10} per continuare l'elaborazione;
- se $i = k$, la sequenza è finita e si passa allo stato S_{11} .

La macchina si prepara a scrivere il valore contenuto in **result** all'indirizzo di memoria **currAddrR**. Vengono quindi posti:

$$o_mem_en = 1, \quad o_mem_we = 1, \quad o_mem_addr = \text{currAddrR}, \quad o_mem_data = \text{result}$$

2.2.12 S_{10} State

Questo stato disattiva l'interazione con la memoria. Controlla il valore di i : se $i < 4$ torna in S_8 , altrimenti va in S_{12} .

L'array **words** viene ricaricato a partire da $i = 4$, in modo tale che l' i -esima parola si trovi nella posizione centrale del vettore: così facendo, per ogni parola, si hanno a disposizione le 3 precedenti e le 3 successive, necessarie per l'applicazione del filtro nel caso pessimo, ovvero il filtro di ordine 5.

Quando $i < 4$, la parola corrente non si trova nella posizione centrale, quindi se il filtro richiede valori come $f[-3]$, $f[-2]$, $f[-1]$, l'indice risultante $jVal + i$ potrebbe essere minore di 0: in tal caso si usa 0 al posto di $f[x]$. Lo stesso vale quando $jVal + i \geq k$ per le ultime parole.

2.2.13 S_{11} State

Quando l'elaborazione è terminata, la macchina va in questo stato. Qui viene posto a 1 il segnale **DONE**. La macchina resta in questo stato finché il segnale **START** non torna a 0, dopodiché ritorna in **Idle** pronta per una nuova elaborazione.

2.2.14 S_{12} State

La macchina va in questo stato quando $i \geq 4$. Questo stato si occupa di aggiornare i registri di memoria per caricare i nuovi valori in **words**. Viene quindi incrementato **currAddrW** e azzerato **counter_words**.

2.2.15 S_{12_wait} State

Qui viene posto:

$$o_mem_addr \leftarrow currAddrW$$

in modo tale che al ciclo successivo, quando la macchina sarà in S_7 per riempire **words**, venga letto in **i_mem_data** il primo valore da inserire nel vettore.

2.3 Descrizione dei processi

2.3.1 Processo 1

È un processo sincrono che lavora ad ogni *rising edge* del clock. Assegna a **currState** il valore precedentemente calcolato e memorizzato in **nextState**. Inoltre, a seconda dello stato corrente, aggiorna i registri con il valore contenuto in **i_mem_data**.

2.3.2 Processo 2

In questo processo viene precalcolato il valore di **nextState** in base a:

$$currState, i_start, counter_coeff, counter_words, i, k.$$

2.3.3 Processo 3

Questo processo gestisce l'interfacciamento con la memoria, impostando le modalità di lettura o scrittura a seconda dello stato in cui si trova la FSM. Inoltre, assegna i valori corretti ai registri **o_mem_addr** e **o_mem_data**.

2.3.4 Processo 4

In questo processo viene applicato il filtro differenziale all'i-esima parola. L'elaborazione avviene in tre fasi:

1. Calcolo della sommatoria

2. Normalizzazione

3. Eventuale saturazione per valori maggiori di 127 o minori di -128

Per i calcoli vengono utilizzate le seguenti variabili locali:

- **currentSomma**: contiene il valore della sommatoria
- **tmpSigned**: contiene il valore di **currentSomma** convertito in signed
- **tmpResult**: contiene il risultato
- **normalized**: contiene il valore normalizzato della somma
- **term1**, **term2**, **term3**, **term4**: valori necessari alla normalizzazione
- **jVal**: indice della sommatoria
- **tmpInt**: contiene il valore di **normalized** convertito a integer

La variabile **currentSomma** viene inizializzata a 0 e incrementata in un ciclo che itera su **jVal**. Nel filtro di ordine 3, **jVal** varia da -2 a $+2$; nel filtro di ordine 5 varia da -3 a $+3$.

Ad ogni ciclo si verifica che la parola $f[x]$ da utilizzare sia contenuta nella sequenza; in caso contrario viene utilizzato il valore 0 al suo posto.

I calcoli avvengono diversamente al variare di i :

caso $i < 4$:

```
currentSomma := currentSomma +  
  (to_integer(signed(coeff(jVal+3))) *  
    to_integer(signed(words(to_integer(signed(i)+to_signed(jVal,i'length))))));
```

essendo che l' i -esima parola non è al centro del vettore.

caso $i \geq 4$:

```
currentSomma := currentSomma +  
  (to_integer(signed(coeff(jVal+3))) *  
    to_integer(signed(words(jVal+3))));
```

essendo che l' i -esima parola è al centro del vettore quindi i valori necessari per i calcoli si trovano intorno ad essa simmetricamente.

Il valore finale di **currentSomma** viene convertito in signed e salvato in **tmpSigned**. Segue la fase di normalizzazione:

- Per il filtro di ordine 3, $n = 12$ approssimato come:

$$\frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}$$

- Per il filtro di ordine 5, $n = 60$ approssimato come:

$$\frac{1}{64} + \frac{1}{1024}$$

Approssimare questi valori riscrivendoli come somme di potenze di due consente di sfruttare la proprietà degli shift a destra, infatti ogni *shift right* corrisponde a dividere il numero per 2.

L'errore di approssimazione più significativo si verifica quando il valore da shiftare è negativo perché ogni shift a destra tronca i bit e questo può introdurre un errore più grande del previsto, per questo se il numero è negativo per ogni shift a destra aggiungiamo 1 al risultato.

Infine, viene applicata la saturazione, che permette di rappresentare il risultato in 8 bit e assegnarlo a `o_mem_data`.

Risultati sperimentali

3.1 Test bench

Il modulo funziona correttamente e supera tutti i test bench eseguiti sia in *behavioral* che in *post-sintesi*, sia *functional* che *timing*. Per generare un gran numero di test bench abbiamo utilizzato un generatore online, oltre che dei test bench significativi pubblicati da alcuni colleghi. Di seguito riportiamo i risultati più rilevanti.

Test bench 25k

Questo test bench è stato ritenuto significativo perché presenta un valore di $k = 25000$, quindi molto grande. Inoltre testa la saturazione siccome molti valori sono fuori range.

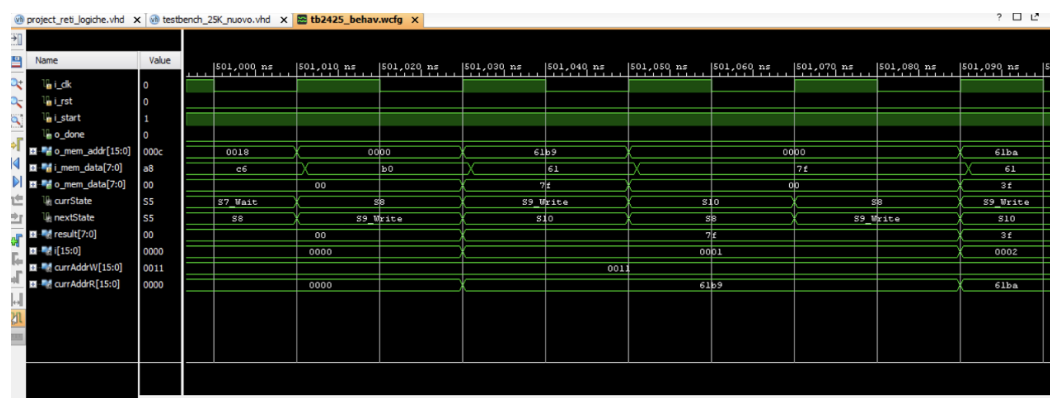


Figura 3.1: Scrittura in memoria del primo valore calcolato

Test bench mark2

Questo test esegue 6 elaborazioni, include diversi reset e testa sia il filtro di terzo ordine che quello di quinto ordine. Viene verificato anche il comportamento in presenza di troncamenti, utilizzando circa 48 KB della RAM disponibile.

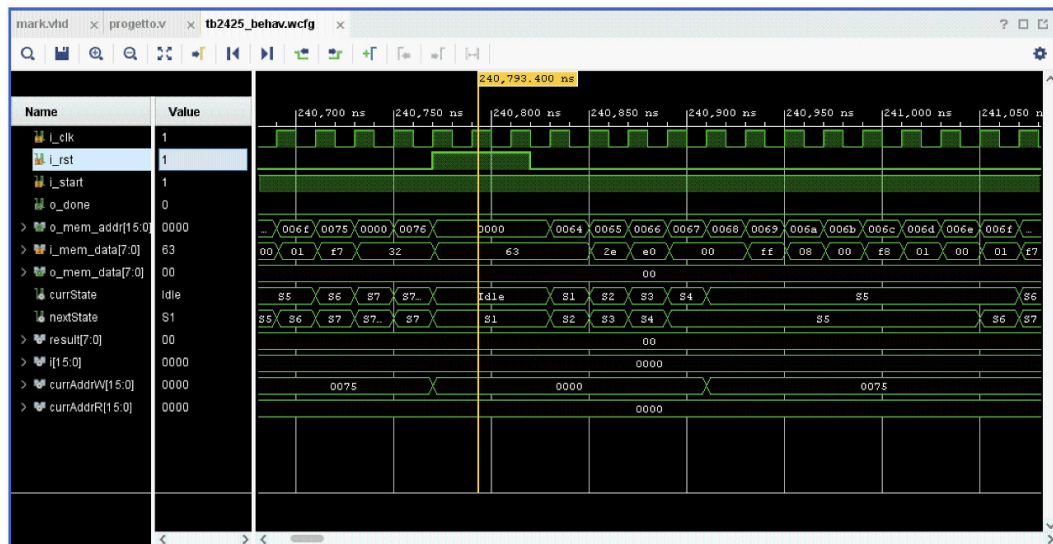


Figura 3.2: Reset asincrono durante un'elaborazione

Questo mostra come tutti i segnali a seguito di un reset asincrono tornino istantaneamente ai loro valori di default. Inoltre è mostrato come dopo di esso inizi una nuova elaborazione.

Test bench new

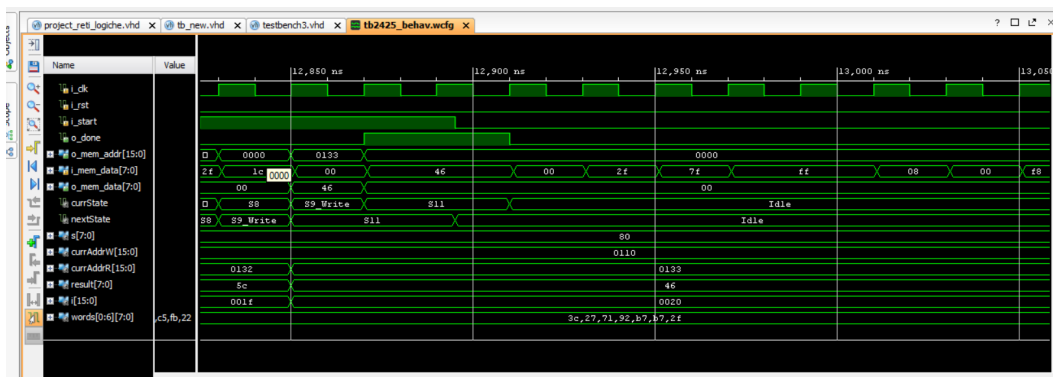


Figura 3.3: Fine elaborazione: segnale DONE a 1, START riportato a 0 e ritorno in Idle ad aspettare un nuovo START

Site Type	Used	Fixed	Available	Util%
Slice LUTs	1898	0	8000	23.73
LUT as Logic	1898	0	8000	23.73
LUT as Memory	0	0	5000	0.00
Slice Registers	227	0	16000	1.42
Register as Flip Flop	227	0	16000	1.42
Register as Latch	0	0	16000	0.00
F7 Muxes	0	0	7300	0.00
F8 Muxes	0	0	3650	0.00

Figura 3.6: slice logic

Conclusioni

Il modulo progettato funziona correttamente sia in *behavioral* che in *post-sintesi* superando tutti i test.

Inizialmente abbiamo progettato la macchina a stati su carta, ragionando principalmente su come gestire le interazioni con la memoria. Successivamente siamo passati alla stesura del codice, nella quale abbiamo cercato di ottimizzare quanto più ci è stato possibile il numero degli stati senza stravolgere la logica su cui avevamo inizialmente lavorato.

Il risultato finale presenta un *Worst Negative Slack* (WNS) positivo, infatti il percorso più lento è molto più veloce del limite imposto di 20ns.

Il progetto inoltre rispetta ampiamente i limiti di risorse dell'FPGA, utilizzando meno di un quarto delle LUT disponibili e una quantità trascurabile di flip flop.

Il dato che risalta principalmente è il numero delle LUTs che è mediamente elevato, sebbene sia ampiamente nei margini. Una motivazione di questo dato è sicuramente la presenza di moltiplicazioni e addizioni in S_8 , in particolare i calcoli relativi a `currentSomma`. Infatti, siccome il progetto non usa DSP dedicati, ogni moltiplicazione viene implementata in LUT che occupano molte risorse.